Hermes: Verified Query Planning over the SPARQL Algebra with Language Agents

Lam Nguyen
Case Western Reserve University
Cleveland, Ohio, USA

ABSTRACT

Text-to-query systems usually emit a whole query, then repair it. Hermes instead answers a question by planning: a language agent builds the query one operator of the SPARQL 1.1 algebra at a time, and execution on the graph after each operator reports the cardinality, how it moved, a sample of rows, and any error. The algebra is formalized the way Lean formalizes mathematics. The agent first states the question as a goal, the operators act as tactics, and a kernel checks scoping, aggregation, and schema types before any plan runs, refusing to finish while a goal is open. Prompt-only generation and a reimplementation of UniQGen sit in the same system, on the same graphs, behind a scorer that reproduces the published Text2SPARQL 2026 numbers and that also reports a strict variant and a reproducible subset. On the 2026 corporate track, with the local model `llama3.1`, the operator-level agent reaches a set F_1 of 0.094 and a recall of 0.300, against 0.064 for UniQGen and 0.017 for one-shot generation, at about seven times UniQGen's model calls. All three are far below the published leaderboard; these are first measurements on one small local model. The verified prover has not been measured yet.

KEYWORDS

SPARQL, knowledge graphs, natural language interfaces, query algebra, execution feedback

PVLDB Reference Format:
Lam Nguyen. Hermes: Verified Query Planning. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lamng3/hermes.

1 INTRODUCTION

A question over a knowledge graph becomes useful once it is a query the graph can run. The usual pipeline asks a model for a complete SPARQL string and, if the string fails to parse or returns nothing, asks again [9, 19]. Agents that explore the graph before answering [2, 8, 21] still write the query as one string. Query optimizers work at a finer grain. They consider one algebraic rewrite at a time, estimate its effect, and keep or discard it [14]. Hermes uses that grain for natural language to SPARQL. The operators of the SPARQL 1.1 algebra [18] are the actions. Execution on the graph is the observation after every action.

The same interface holds methods that were not written as planners. A one-shot prompt and UniQGen's Chase and Backchase [4, 19] are policies over this space. They share graph stores, a normalized suite of benchmarks, and one scorer, so a later comparison can hold the model and the call budget fixed. This paper records the design and the first finished runs. It does not answer the comparison. Four questions organize the rest of the work.

(1) Does operator-level execution feedback beat one-shot generation with repair, at equal model size and equal LLM-call budget?
(2) Which part of the observation matters: cardinality, the change in cardinality, row samples, or errors?
(3) Can a search policy over the same actions match a prompted policy with fewer model calls?
(4) How far do results on the Text2SPARQL corporate graph transfer to DBpedia and Wikidata?

The system is available at the artifact URL above, under CC BY 4.0. Benchmark files keep the licenses of their sources.

2 THE SPARQL ALGEBRA AS ACTIONS

A query is an immutable logical plan: triple patterns, filters, optional groups, unions, minus groups, bindings, grouping, aggregates, having-clauses, ordering, the projection, distinctness, limit, offset, and the query form (`SELECT` or `ASK`). Rendering the plan is deterministic. An action is a pure function from one plan to the next. Table 1 lists them against the SPARQL 1.1 algebra [13, 18]. Two further actions read the graph and leave the plan unchanged. `lookup` finds entities whose label contains a string. `inspect` lists predicates around an entity or a class. `undo` restores the previous plan. `finish` stops the policy.

The explorer applies an action and runs the resulting plan. The observation records whether the query parsed, the solution count, the previous count when it changed, the projected columns, a sample of rows (five by default), and an error string when the step failed. Cardinality is a `COUNT(*)` over the rendered pattern; the sample is the same pattern under `LIMIT`. An `ASK` plan reports the Boolean in place of a count. `preview` returns that observation and does not commit the action, which is the planner's analogue of costing a rewrite before choosing it. A step that fails to parse, or that the kernel rejects (Section 3), is not committed and is not sent to the store.

Table 2 is the shape of that feedback, from a walk over a small example graph. It is an illustration, not a benchmark result.

Any policy can drive the explorer. The algebra agent is one such policy. Each turn it sees the question, the vocabulary, the current

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Actions and the SPARQL 1.1 operators they apply.

Action	Algebra operator
`triple`	BGP, extended by join
`filter`	Filter
`optional`	LeftJoin
`union`	Union
`minus`	Minus
`bind`	Extend
`group`	Group
`aggregate`	Aggregation
`having`	Filter on groups
`order`	OrderBy
`project`	Project
`distinct`	Distinct
`limit`, `offset`	Slice
`ask`	ASK query form
`remove`	retract a triple or a filter

Table 2: Explorer feedback on a small example graph.

Action	Observation
`triple ?city ex:hosted ?g`	19 solutions
`triple ?city ex:country ex:USA`	19 → 8
`triple ?g ex:season "Winter"`	8 → 4
`distinct`	4 → 3

Table 3: The prover, read as a tactic language.

Lean	Hermes
Theorem statement	Goal: variables, kind, type, cardinality
Tactic	Algebra action
Local context Γ	Variable ↦ inferred classes
Kernel	`Kernel.check(plan, goal)`
Unknown identifier	Term outside the schema namespaces
Type mismatch	Domain, range, or a literal as subject
No goals left	No errors and no open goals

plan as SPARQL, and the observation after every earlier action, and it replies with one JSON action. The loop ends on `finish` or after 15 steps, and the result is the last plan that ran. An action identical to the previous one is not executed; the model is told to choose another. UniQGen's Chase and Backchase, in these terms, is a beam search over `remove` and `triple`.

3 A KERNEL IN THE STYLE OF LEAN

The algebra agent will execute any plan that parses. The prover puts a kernel in front of that execution. The design follows interactive theorem provers such as Lean [3]: the question is a goal, algebra actions are tactics, and the kernel checks each step before the graph sees it. Table 3 is the correspondence.

The model states the goal first, for example one department, or a number, or a yes-or-no answer. The kind is `entity`, `literal`, `number`, or `boolean`. A class may be attached, and `many` is false when a single answer is required. Typing comes from the schema [17]. A triple `?x a C` gives `?x : C`. A triple `?x p ?y` gives `?x` the domain of `p` and `?y` its range. Classes are compared up to `rdfs:subClassOf`. Under the open-world assumption, a class conflict is a warning. Using a literal where an entity is required is an error, as is a literal in subject position, an unknown class or property in a namespace the schema covers, a `BIND` onto an already bound variable, an unbound variable in `FILTER`, `GROUP BY`, `HAVING`, `ORDER BY`, or `SELECT`, and a projected variable that is neither grouped nor aggregated.

Open goals are what `finish` still has to discharge: an `ASK` for a question that wants values, or the reverse; an answer variable that is not bound or not projected; a variable the goal calls a number that is not numeric; an entity variable typed as a literal; a required class that the variable's types do not fall under; extra projected variables; and, once the plan has run, more than one solution when the goal asked for one. `finish` is refused while any error or open goal remains. The prompt shows the proof state each turn: the typing context and the open goals. Ill-typed steps increment a rejection counter and never reach the store.

The analogy has a limit, and it locates the hard part. LLM provers for Lean [22] search for tactics against a theorem a person wrote, and a proof the kernel accepts is correct. Here the goal is written by the model from the question, which is autoformalization [20]. The kernel guarantees that an accepted plan is well formed, typed against the schema, and shaped like its goal. It does not guarantee that the goal is the question: a well-typed query can still answer a different one. Execution feedback, the analogue of evaluating terms in Lean, is the remaining evidence for that step.

The prover is the algebra agent with this kernel installed. It has an implementation and tests. It does not yet have an experiment configuration, so Section 6 does not report it.

4 DESIGN

Hermes is a small runtime around that loop. A system is any callable from a question, a knowledge graph, and an optional context to a SPARQL string. The context can carry text, example question–query pairs, and, for ablations only, reference answers. Four systems are registered: `training-free`, `uniqgen`, `algebra-agent`, and `prover`. Another package can add one through the `hermes.systems` entry point. Each receives the language model and the graph by construction, and experiment files pass constructor arguments (`beam_width`, `max_steps`, and so on) without code changes.

The one-shot system prompts a single draft from the ontology vocabulary, passes it through a fixed sequence of stages (today only a guardrail that declares known prefixes the draft uses but omits), and, if the draft does not parse, makes one further model call with the parser error. UniQGen follows the three stages of Wang et al. [19], rendered to SPARQL on whatever store is configured. Entities are linked by label and their neighborhood is collected; one model call proposes a constraint table, and constraints with no match in the graph are pruned; Chase and Backchase then searches for a query that covers the reference answers and a minimal query that returns only those answers. Two choices differ from that paper, and they matter when reading Section 6. The query text is rendered from the constraints rather than by a second model, and candidates are scored by comparing answer sets rather than by asking a model. That leaves two model calls per question, the budget the UniQGen

Table 4: Normalized benchmarks. Question counts are source questions; Text2SPARQL items are scored per language.

Benchmark	Questions	Graph
CK26	50	Corporate KG, local
DB26	50	DBpedia, organizers' endpoint
CK25	50	Corporate KG, local
DB25	100	DBpedia
QALD-10 [16]	394	Wikidata, live
QALD-9-plus [12]	150	DBpedia, live

Table 5: Finished runs, official metrics except the strict F_1 column. Model: `llama3.1`. The gold-oracle row is an ablation, not a result.

	System	F_1	P	R	F_{ndcg}	strict
CK26	training-free	0.017	0.036	0.015	0.018	0.017
	uniqgen	0.064	0.061	0.142	0.064	0.064
	uniqgen, gold oracle	0.086	0.082	0.229	0.086	0.086
	algebra-agent	0.094	0.101	0.300	0.094	0.094
DB26	training-free	0.017	0.038	0.011	0.017	0.017

paper reports. Aggregates, `ORDER BY`, and `LIMIT` are not handled. A gold-oracle variant, which places reference answers in the context, is configured for ablation and is not a result.

Stores expose one method, `query`, returning SPARQL 1.1 JSON. The same system therefore runs on local RDF files in Oxigraph [11] (the default) or rdflib, or on any SPARQL endpoint. A cache wraps the store, memoizes results, and counts executions. The model client is likewise cached on disk, so a rerun replays the same completions. The default model is a local Ollama model; a config may name any OpenAI-compatible endpoint instead.

An experiment is a YAML file: benchmark, system, model, and optional language filter, item limit, and few-shot pool. A run predicts, executes, and scores each item, and can resume after an interruption. It writes the config, per-item predictions and traces, aggregate metrics, and a provenance record: model, model calls, cache hits, graph executions, git commit, and wall time. The same process can be served on the Text2SPARQL challenge API, so a system can be submitted to the organizers' client.

5 BENCHMARKS, METRICS, AND CAVEATS

Table 4 is the normalized suite. Each benchmark is pinned to a source commit and a checksum. Questions carry an id of the form used by the official scorer, a language, the gold query, feature tags (`COUNT`, `GROUP`, `ORDER`, `ASK`, and the rest), and whether order matters. Gold answers are stored in the official layout: every value of every variable flattened into one set. QALD answers are converted into that layout. Adapters isolate the source formats, so a new benchmark is a loader plus a manifest.

The scorer follows the Text2SPARQL pipeline [1, 10]. Every binding value is a document of relevance 1. Per question it reports set precision, set recall, and set F_1. Where the organizers mark order as significant it also reports nDCG. The headline `set_F_ndcg` is nDCG on those questions and set F_1 elsewhere. Rescoring every published Text2SPARQL 2026 prediction reproduces all 5,024 per-question and average numbers in the official files exactly; `experiments/check_scorer.py` repeats the check from the pinned sources. Runs report three aggregates side by side, because they answer different questions: the official metrics on every item, a strict variant, and the official metrics on the items whose gold query reproduces on our store.

Several properties of the data change how a number should be read.

(1) Forty-nine of the fifty CK26 gold queries appear verbatim in CK25, on the same graph. Neither edition is a development set for the other, and few-shot examples drawn from one leak the other. Runs record that overlap as contaminated examples. DB25 and DB26 share no gold query.
(2) Official ASK scoring is degenerate. The scorer treats an ASK answer as a document named `true`. A gold `true` scores 1.0 whatever the system returns; a gold `false` scores 0.0 for every system. The strict variant scores Booleans by exact match.
(3) Official nDCG ignores result order. Every returned value has the same score, so trec_eval orders values by their text. The strict variant uses the order the system returned.
(4) Official `set_F_ndcg` counts the mean nDCG of a scope as one extra item in the average. Hermes reproduces that, so its official column matches the leaderboard. The strict column is a plain mean.
(5) CK gold answers were computed on Tentris. On Oxigraph, 43 of 50 CK26 gold queries return the gold answers. The other seven break ties under `LIMIT` differently, format decimals differently, or call `xsd:int()`, which is not a SPARQL 1.1 function. Those items are flagged `gold_reproduces`. On rdflib, 42 of 50 agree. On the organizers' DBpedia endpoint, all 50 DB26 gold queries agree.
(6) QALD-9-plus targets DBpedia 2016-10, and QALD-10 was answered on Wikidata in 2022. The live endpoints have changed since, so a score against today's endpoint is not a score against the original graph.

Comparable numbers follow from these facts. Official metrics on all items are what can be set next to published entries. Strict and reproducible figures stay next to them. A run with `leak_gold` or with contaminated examples is an upper bound. The model is part of the result: several published entries used larger API models, and a run on local `llama3.1` is a lower bound for a method.

6 PRELIMINARY RESULTS

Five runs have finished, all with local Ollama `llama3.1` and all with `contaminated_examples` at 0: four on CK26 (50 questions in English and German, 100 items) and the one-shot system on DB26 (50 questions in English and Spanish, 100 items). Table 5 rounds the run files to three decimal places, and Table 6 gives the cost of each run. An error is an item for which no query was produced or the query failed to run.

Table 6: Cost per run (100 items each).

	System	Calls	Executions	p50 (s)	Errors
CK26	training-free	116	136	3.0	24
	uniqgen	198	667	15.1	41
	algebra-agent	1,451	422	24.4	4
DB26	training-free	103	144	3.2	9

Table 7: Published Text2SPARQL 2026 CK26 scores (`set_F_ndcg`), organizer evaluation.

Entry	F_{ndcg}
SPARQL-LLM	0.832
INFAI-ETI-AND-FRIENDS-A	0.633
ADFR	0.513
INFAI-ETI-AND-FRIENDS-C	0.452
LIBER-AI-CLAUDE	0.335
IRIS	0.217
LIBER-AI-QWEN	0.206
INFAI-ETI-AND-FRIENDS-B	0.157

The algebra agent has the highest F_1 and the highest recall, and it fails on 4 items where UniQGen fails on 41: a policy that builds the plan one operator at a time almost always ends with a query that runs. It averaged 13.9 steps per item, 37 items scored above zero, and 4 were answered exactly. The comparison is not at equal budget. The agent made about 14.5 model calls per item, UniQGen about 2, so these numbers do not yet answer the first research question; a budget-matched comparison is the next experiment. The gold-oracle ablation replays UniQGen's cached constraint tables and changes only the reference answers, so its gain over the LLM oracle (0.064 to 0.086) isolates the oracle; it still stays below the agent, because 41 items fail before the search starts, when the constraint table matches nothing in the graph. Restricting to the 86 CK26 items whose gold query reproduces on Oxigraph changes no headline number by more than 0.004, and the strict and official columns agree, since none of these runs gets credit from the ASK or nDCG quirks. On DB26, where the vocabulary is not shown to the model, the one-shot system matches its CK26 score.

Table 7 is the published CK26 leaderboard, for scale. It is not a paired comparison. The organizers scored those systems on Tentris, the model behind each entry is not controlled here, and our runs execute on Oxigraph. Read against that board, every local run is below the weakest published entry; the published entries also use larger models. The prover, the DB26 agent, QALD, and model scaling have no finished run yet.

7 RELATED WORK

Text-to-SQL.. Liu et al. survey LLM-based text-to-SQL by pipeline stage: schema linking and example selection before translation, self-correction and execution-guided selection after it [9]. Alpha-SQL builds SQL step by step with Monte Carlo tree search, using an LLM to propose construction actions and a self-supervised reward to score candidates [7]. Its actions are reasoning steps toward SQL text rather than operators of the relational algebra, and no schema typing is enforced between steps.

Grounded KBQA and agents. Pangu lets a symbolic agent grow valid plans over the knowledge base while a language model only discriminates between them [5]; it is the closest prior design to incremental, execution-backed plan building, but its plans are its own logical forms over Freebase, and its language model scores candidates instead of reading execution feedback. Middleware tools give an LLM functions such as relation and neighbor lookups over large knowledge bases [6], and Interactive-KBQA exposes generic KB APIs across multi-turn interaction [21]. Their action sets are task-specific functions, not the query algebra.

Natural language to SPARQL.. SPINACH explores Wikidata by executing SPARQL as it reasons and reports strong results on QALD-9-plus and QALD-10 [8]; ARUQULA builds on it for the first Text2SPARQL challenge [2, 15]. These agents use execution as feedback, but each step is a whole query string. UniQGen [19] searches over a table of constraints with Chase and Backchase [4] against reference answers. Hermes differs in three ways: the action space is the SPARQL algebra itself; each operator is observed (and can be previewed) before it is kept; and a kernel types each step against the RDF schema before it runs, so methods such as UniQGen become policies over one space.

8 CONCLUSION

Hermes makes natural language to SPARQL a verified planning problem: the actions are the SPARQL algebra, the feedback is execution, and a kernel checks every step against a stated goal. A Lean-style kernel types each step against the schema and holds the goal open until the plan discharges it. The one-shot baseline and UniQGen already run behind that interface, on benchmarks whose official scores the scorer reproduces, with the ASK, nDCG, overlap, and engine caveats kept visible beside the official column. On CK26 with `llama3.1`, the operator-level agent reaches 0.094 set F_1, against 0.064 for UniQGen and 0.017 for one-shot generation, and almost never ends without a runnable query, though at a larger call budget. The measurements that would answer the four questions in the introduction — a budget-matched comparison, the prover, the feedback ablations, and the move to DBpedia and Wikidata — are next.

REFERENCES

[1] AKSW. 2026. The Text2SPARQL Challenge 2026: Datasets, Results, and Evaluation Client. Challenge site and result repository. https://github.com/AKSW/text2sparql.aksw.org

[2] Felix Brei, Lorenz Bühmann, Johannes Frey, Daniel Gerber, Lars-Peter Meyer, Claus Stadler, and Kirill Bulert. 2025. ARUQULA – An LLM based Text2SPARQL Approach using ReAct and Knowledge Graph Exploration Utilities. In *Proceedings of the First International TEXT2SPARQL Challenge 2025 (CEUR Workshop Proceedings)*, Vol. 4094. CEUR-WS.org, Aachen, Germany, 49–63. https://ceur-ws.org/Vol-4094/

[3] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction — CADE 28 (LNCS)*, Vol. 12699. Springer, Cham, 625–635.

[4] Alin Deutsch, Lucian Popa, and Val Tannen. 1999. Physical Data Independence, Constraints, and Optimization with Universal Plans. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, San Francisco, CA, USA, 459–470.

[5] Yu Gu, Xiang Deng, and Yu Su. 2023. Don't Generate, Discriminate: A Proposal for Grounding Language Models to Real-World Environments. In *Proceedings of*

the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Toronto, Canada, 4928–4949. https://doi.org/10.18653/v1/2023.acl-long.270

[6] Yu Gu, Yiheng Shu, Hao Yu, Xiao Liu, Yuxiao Dong, Jie Tang, Jayanth Srinivasa, Hugo Latapie, and Yu Su. 2024. Middleware for LLMs: Tools Are Instrumental for Language Agents in Complex Environments. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 7646–7663. https://doi.org/10.18653/v1/2024.emnlp-main.436

[7] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *Proceedings of the 42nd International Conference on Machine Learning (PMLR)*, Vol. 267. PMLR, Vancouver, Canada, 36810–36830.

[8] Shicheng Liu, Sina Semnani, Harold Triedman, Jialiang Xu, Isaac Dan Zhao, and Monica Lam. 2024. SPINACH: SPARQL-Based Information Navigation for Challenging Real-World Questions. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 15977–16001. https://doi.org/10.18653/v1/2024.findings-emnlp.938

[9] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? arXiv:2408.05109. https://arxiv.org/abs/2408.05109

[10] National Institute of Standards and Technology. 2024. trec_eval. Information retrieval evaluation software. https://github.com/usnistgov/trec_eval

[11] Oxigraph contributors. 2026. Oxigraph. RDF graph database. https://github.com/oxigraph/oxigraph

[12] Aleksandr Perevalov, Dennis Diefenbach, Ricardo Usbeck, and Andreas Both. 2022. QALD-9-plus: A Multilingual Dataset for Question Answering over DBpedia and Wikidata Translated by Native Speakers. In *2022 IEEE 16th International Conference on Semantic Computing (ICSC)*. IEEE, Piscataway, NJ, USA, 229–234. https://doi.org/10.1109/ICSC52841.2022.00045

[13] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and Complexity of SPARQL. *ACM Transactions on Database Systems* 34, 3 (2009), 16:1–16:45. https://doi.org/10.1145/1567274.1567278

[14] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 23–34. https://doi.org/10.1145/582095.582099

[15] Sebastian Tramp, Marcos Gôlo, Edgard Marx, and Paulo Viviurka do Carmo (Eds.). 2025. *Proceedings of the First International TEXT2SPARQL Challenge 2025, co-located with Text2KG at ESWC 2025*. CEUR Workshop Proceedings, Vol. 4094. https://ceur-ws.org/Vol-4094/

[16] Ricardo Usbeck, Xi Yan, Aleksandr Perevalov, Longquan Jiang, Julius Schulz, Angelie Kraft, Cedric Moeller, Junbo Huang, Jan Reineke, Axel-Cyrille Ngonga Ngomo, Muhammad Saleem, and Andreas Both. 2023. QALD-10 — The 10th Challenge on Question Answering over Linked Data. Semantic Web journal, submission 3357; dataset at https://github.com/KGQA/QALD-10. https://www.semantic-web-journal.net/node/3357

[17] W3C RDF Working Group. 2014. RDF Schema 1.1. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf-schema/

[18] W3C SPARQL Working Group. 2013. SPARQL 1.1 Query Language. W3C Recommendation, 21 March 2013. https://www.w3.org/TR/sparql11-query/

[19] Mengying Wang, Nicolaas Jedema, Rahul Pandey, RaviKiran Krishnan, Jens Lehmann, and Yinghui Wu. 2026. Graph Query Generation with Constraint-guided Large Language Agents. arXiv:2605.00845. https://arxiv.org/abs/2605.00845

[20] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, Vol. 35. Curran Associates, Inc., Red Hook, NY, USA. https://doi.org/10.52202/068431-2344

[21] Guanming Xiong, Junwei Bao, and Wen Zhao. 2024. Interactive-KBQA: Multi-Turn Interactions for Knowledge Base Question Answering with Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 10561–10582. https://doi.org/10.18653/v1/2024.acl-long.569

[22] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*. Curran Associates, Inc., Red Hook, NY, USA.